Homework 1:
Python Practice and Linear Algebra
CPE 486/586

Instructor: Rahul Bhadani

Due: Sept 13, 2026, 11:59 PM
110 points

You are allowed to use a generative model-based AI tool for your assignment. However, you must submit an accompanying reflection report detailing how you used the AI tool, the specific query you made, and how it improved your understanding of the subject. You are also required to submit screenshots of your conversation with any large language model (LLM) or equivalent conversational AI, clearly showing the prompts and your login avatar. Some conversational AIs provide a way to share a conversation link, and such a link is desirable for authenticity. Failure to do so may result in actions taken in compliance with the plagiarism policy.

Additionally, you must include your thoughts on how you would approach the assignment if such a tool were not available. Failure to provide a reflection report for every assignment where an AI tool is used may result in a penalty, and subsequent actions will be taken in line with the plagiarism policy.

Submission instruction:

You may either complete your entire homework on Python Notebook and submit a Google Colab link or you may choose a combination of a pdf and Google Colab Notebook. However, you must provide **publicly accessible** Google Colab URL on Canvas.

For .pdf on Canvas, follow the format `CPE486586-LastFirst-HW-XX`. For example, if your name is Sam Wells, your file name should be `CPE486586-WellsSam-HW-XX.pdf`.

Your submission must contain your name and UAH Charger ID or your UAH email address. Please number your pages as well.

Please write down your unique keyword corresponding to your package.

Note: Python community has adopted following import convention that we will follow for the rest of the course:

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns
import statsmodels as sm
```

For this portion of the Homework, complete your implementation entirely in the notebook and for a portion, you will contribute to your package and demonstrate its usage in your notebook.

1 Adding Two Vectors (10 Points)

What happens when try to add the following two vectors:

1. ```
   v = np.array([[1,5,6]])
   w = np.array([[2,4],[6,7]])
   ```

   What is the output of `v+w`?

2. ```
   v  = np.array([[1,5]])
   w = np.array([[2,4],[6,7]])
   ```

   What is the output of `v+w`?

For both of the above questions, you need to describe why they work, or don't work, and what's the final addition?

Answer

1. We get error

   ValueError: operands could not be broadcast together with shapes (1,3) (2,2)

   This error happens because in column dimension size of 3 cannot be broadcasted to size of 2. Row dimension is fine because size of 1 can be broadcasted to

2. Output:

```
array([[ 3,  9],
       [ 7, 12]])
```

Here row dimension and columns both can be broadcasted due to one being multiple of the others.

2 Inverse of a Matrix (10 Points)

Use Numpy linear algebra library to find the inverse of

```
np.array([[4,6,7,12],[8,9,0,-5], [4,1,0,0],[3,2,0,1]])
```

rounded to the two places after the decimal. Use rounding directly in the numpy as report the result.

Answer

```
p = np.array([[4,6,7,12],[8,9,0,-5], [4,1,0,0],[3,2,0,1]])
p_inv = np.linalg.inv(p)
```

which gives answer as

```
[[ 0.       -0.01886792  0.35849057 -0.09433962]
 [-0.        0.0754717  -0.43396226  0.37735849]
 [ 0.14285714  0.10781671  0.52291105 -1.17520216]
 [ 0.       -0.09433962 -0.20754717  0.52830189]]
```

3 The Longest Substring (10 Points)

Write a function `LongestSubstring` in Python that takes a Python string as an argument. Your function should perform the following task:

1. If the string is empty print 'empty string' and return.

2. If the string length exceeds 20 characters, print 'maximum length exceeded' and return.

3. Otherwise, find the long substring in the given string that contains same characters. For example, if the input string is '2242777232888823', then the output should be '8888'.

4. Write code to use the function `LongestSubstring` as well.

The implementation doesn't need to be the most efficient implementation, and you may choose to use nested `for` loops as well.

Answer

```
    def LongestSubstring(s):
    # Check if the string is empty
    if not s:
        print("empty string")
        return

    # Check if the string length exceeds 20 characters
    if len(s) > 20:
        print("maximum length exceeded")
        return

    max_substring = ""
    current_substring = ""

    # Iterate through the string to find
    # the longest substring with the same characters
    for i in range(len(s)):
        if i == 0 or s[i] == s[i - 1]:
            current_substring += s[i]
        else:
            if len(current_substring) > len(max_substring):
                max_substring = current_substring
            current_substring = s[i]

    # Final check in case the longest
    # substring is at the end of the string
    if len(current_substring) > len(max_substring):
        max_substring = current_substring

    # Print the result
    print(max_substring)
# Example usage
LongestSubstring("2242777232888823")  # Output: '8888'
LongestSubstring("")                # Output: 'empty string'
LongestSubstring("1" * 24)           # 'maximum length exceeded'
LongestSubstring("aaabbbbaacccdd")    # Output: 'bbbb'
```

4 Data and Time in Python (5 Points)

The built-in Python `datetime` module provides `datetime`, `date`, and `time` types. The `datetime` type combines the information stored in date and time.

Write a Python program that creates an empty file with the file name in the format ML-HW01_%Y-%m-%d-%H-%M.txt, where %Y is substitute for current year, %m is the substitute for current month, %d is substitute for current day, %H is the substitute for current hour, and %M is the substitute for current minute. One possibility can be a file name **ML-HW01-2011-10-29-20-30.txt**.

Answer

```
import datetime

# Get the current date and time
time_now = datetime.datetime.now()


# Formatting to required filename
required_filename = time_now.strftime("ML-HW01-%Y-%m-%d-%H-%M.txt")

# This creates an empty file with the required filename
with open(required_filename, "w") as file:
   pass

print(f"{required_filename} is the empty file created.")
```

5 Working with Data Types in Python (5 Points)

In Python, there are some special data types such as table, and lists. A tuple is a fixed-length, immutable sequence of Python objects which, once assigned, cannot be changed. An example is `(2, 4, 5)`. On the other hand, lists are variable length and their contents can be modified in place. An example is `[1, 4, 5]`. Write a Python function that takes an argument as a list or a tuple, and returns a tuple containing the number of element the list or a tuple has, the maximum value, and a minimum value.

Answer

```python
# x should be a list or tuple input
def lenmaxmin(x):

    # Calculating the length, maximum, and minimum value of the list/tuple
    num_elements = len(x)
    maxi = max(x)
    mini = min(x)

    # This returns the value as tuple
    return (num_elements, maxi, mini)


print(lenmaxmin((12,6,4,65)))
print(lenmaxmin([311,45,9]))
print(lenmaxmin([311,45,9, 74, 10222, 89, -93]))
```

6 Slicing (5 Points)

You can select sections of most sequence types by using slice notation, which in its basic form consists of `start:stop` passed to the indexing operator `[]`:

In this part of the homework, you will create a 4×4 tensor using PyTorch package. Initialize your tensor randomly with only 0s and 1s. Tensor is a generalized name of a multi-dimensional array or matrix in machine learning domain. Note that this concept of tensor differs from one seen in Mathematics textbooks. Write a Python program to slice the last column of a 4×4 tensor.

Answer

```python
import torch

# This creates a random tensor of size 4 x 4 consisting of 1s and 0s
rand_tensor = torch.randint(0, 2, (4, 4))
   # or tensor[:,3]. This slices the last column of the 4 x 4 tensor
last_column = rand_tensor[:, -1]
# convert to a column vector
last_column = last_column.reshape(-1, 1)
print(f"The tensor is:\n{rand_tensor}\n")
print(f"The last column is:\n{last_column}")
```

7 Dictionary (5 Points)

Dictionary is a key-value pair data-types in Python. Create a dictionary in Python consisting of two letter state code of the neighboring states of Alabama as the key and their capital as the value.

Answer

```python
#Dictionary of Alabama's neighbouring states'
# two letter code as keys and the states' capital as value
AL_neighbouring_states = {"MS": "Jackson", "GA":"Atlanta",
   "FL":"Tallahassee", "TN":"Nashville"}
print(AL_neighbouring_states)
print(AL_neighbouring_states["FL"])
print(AL_neighbouring_states["MS"])
print(AL_neighbouring_states["GA"])
print(AL_neighbouring_states["TN"])
```

8 Vector and Matrix Operations (10 Points)

Let $\mathbf{u} = [4, -1, 3]^T$, $\mathbf{v} = [2, 0, 5]^T$.

1. Compute the inner product $\mathbf{u} \cdot \mathbf{v}$.
2. Compute the angle between the vectors in degrees.

3. Implement the calculation in NumPy and verify the result.

Answer

8.1 Compute the inner product u · v

$$\mathbf{u} \cdot \mathbf{v} = 4 \cdot 2 + (-1) \cdot 0 + 3 \cdot 5 = 8 + 0 + 15 = \boxed{23}$$

8.2 Compute the angle between the vectors

The cosine of the angle θ between two vectors is given by:

$$\cos(\theta) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \cdot \|\mathbf{v}\|}$$

First, compute the magnitudes:

$$\|\mathbf{u}\| = \sqrt{4^2 + (-1)^2 + 3^2} = \sqrt{16 + 1 + 9} = \sqrt{26}$$

$$\|\mathbf{v}\| = \sqrt{2^2 + 0^2 + 5^2} = \sqrt{4 + 0 + 25} = \sqrt{29}$$

Now compute cosine:

$$\cos(\theta) = \frac{23}{\sqrt{26} \cdot \sqrt{29}} \approx \frac{23}{\sqrt{754}} \approx \frac{23}{27.46} \approx 0.8376$$

$$\theta = \cos^{-1}(0.8376) \approx \boxed{33.4^\circ}$$

```python
import numpy as np

u = np.array([4, -1, 3])
v = np.array([2, 0, 5])
dot = np.dot(u, v)  # 23

cos_theta = dot / (np.linalg.norm(u) * np.linalg.norm(v))
angle_deg = np.degrees(np.arccos(cos_theta))  # ~33.4 degrees
```

The NumPy computation confirms the manual result.

9 Matrix Multiplication and Rank (10 Points)

Given matrices:

$$A = \begin{bmatrix} 2 & 4 \\ 0 & 3 \\ 1 & -1 \end{bmatrix}, \quad B = \begin{bmatrix} 1 & 2 & 3 \\ 0 & -1 & 1 \end{bmatrix}$$

1. Compute AB.
2. Determine the rank of matrix A.
3. Use NumPy to compute the rank.

Answer

9.1 Compute AB

$$AB = A \cdot B = \begin{bmatrix} 2\cdot 1 + 4\cdot 0 & 2\cdot 2 + 4\cdot(-1) & 2\cdot 3 + 4\cdot 1 \\ 0\cdot 1 + 3\cdot 0 & 0\cdot 2 + 3\cdot(-1) & 0\cdot 3 + 3\cdot 1 \\ 1\cdot 1 + (-1)\cdot 0 & 1\cdot 2 + (-1)\cdot(-1) & 1\cdot 3 + (-1)\cdot 1 \end{bmatrix} = \begin{bmatrix} 2 & 0 & 10 \\ 0 & -3 & 3 \\ 1 & 3 & 2 \end{bmatrix}$$

9.2 Rank of A

Since the rows of A are linearly independent, Rank $= \boxed{2}$.

9.3 NumPy Verification

```
import numpy as np
A = np.array([[2, 4], [0, 3], [1, -1]])
rank = np.linalg.matrix_rank(A)    # returns 2
```

10 Eigenvalue Decomposition with PyTorch (10 Points)

Let $M = \begin{bmatrix} 2 & 1 \\ 1 & 3 \end{bmatrix}$.

1. Use PyTorch to compute the eigenvalues and eigenvectors.
2. Verify that $Mv = \lambda v$ for one eigenpair.

Answer

10.1 Compute eigenvalues and eigenvectors

Characteristic polynomial:

$$\det(M - \lambda I) = (2-\lambda)(3-\lambda) - 1 = \lambda^2 - 5\lambda + 5 \Rightarrow \lambda = \frac{5 \pm \sqrt{5}}{2}$$

10.2 PyTorch Verification

```
import torch
M = torch.tensor([[2.0, 1.0], [1.0, 3.0]])
eigvals, eigvecs = torch.linalg.eigh(M)
Mv = M @ eigvecs[:, 0]
lambda_v = eigvals[0] * eigvecs[:, 0]
```

$Mv = \lambda v$ is verified numerically.

11 Solving System of Linear Equations(10 Points)

Consider a system of linear equations:

$$\begin{aligned} 4x + 3y + 2z &= 25 \\ -2x + 2y + 3z &= -10 \\ 3x - 5y + 2z &= -4 \end{aligned} \tag{1}$$

In a matrix form, they can be written $AX = B$. Identify A, and B.

Write a Python function `linearsolve` that takes two arguments: (a) matrix A; (b) vector B and return the solution vector X. You may implement your solution using Numpy or PyTorch.

Answer

$$A = \begin{bmatrix} 4 & 3 & 2 \\ -2 & 2 & 3 \\ 3 & -5 & 2 \end{bmatrix}$$
$$B = \begin{bmatrix} 25 \\ -10 \\ -4 \end{bmatrix} \tag{2}$$

```
import torch
A = torch.tensor([[4, 3, 2], [-2, 2, 3], [3, -5, 2]], dtype=torch.float32)
B = torch.tensor([[25], [-10], [-4]], dtype=torch.float32)
def linearsolve(A, B):
    """
    Args:
        A (torch.tensor): System Matrix
        B (torch.tensor): Right Hand Side

    Returns:
        Solution Vector
    """
    X = torch.linalg.solve(A, B)
    return X

X = linearsolve(A, B)
print(X)
```

Solution is: $X = [5, 3, -2]$.

Through the next few questions, you will make a contribution to your Python package with your unique keyword name.

12 Elementary Operations on Matrices (40 Points)

Create a subpackage called `matrix` in your package. Inside the subpackage, you will be creating a module `elementary.py` with the following requirements usin PyTorch based arrays and matrices:

1. Write a function `rowswap` that takes a matrix as input, along with the index of source row and the target row, and swap the content of source row with the target row.

2. Write a function `rowscale` that takes a matrix as an input, along with the index of source row and scaling factor, and perform row-scaling and return the new matrix. The scaling factor can be any real number, positive or negative.

3. Write a function `rowreplacement` to perform $jR_i + kR_j$ that takes a matrix as an input, first row, the second row, the scaling factors j and k. You may use function `rowscale` for scaling operation.

4. Please refer to the discussion on Row Echelon Form from Chapter 03 around slide 74 onwards. Pivot element is the element in each row of a matrix below which there is all zero (but not above). We also consider the reduced Row Echelon Form where each Pivot element is 1. As an example, in the first row, the first column it is always one and below it, all zero.

   Write a function called `rref` that takes a matrix as an input. The task is to (i) Make the pivot element in the every row 1, (ii) make all the element below pivot element as 0. Here, you need to use if-else or other programming construct to determine which rows to swap, and what should be the scaling factor. The function will be return the reduced row echelon form of the matrix. You can use three functions developed earlier to assisting you in implementing `rref`.

5. Explain your idea of implementing `rref` in paragraphs.

6. You must also add `__init__.py` with appropriate import code to make sure your subpackage is a valid and importable subpackage.

7. Build your package, commit to GitHub and publish to PYPI after properly incrementing the version number, install the package in your Google Colab notebook using `pip install` command.

8. To test your program consider the input matrix as

$$\begin{bmatrix} 1 & 3 & 0 & 0 & 3 \\ 0 & 0 & 1 & 0 & 9 \\ 0 & 0 & 0 & 1 & -4 \end{bmatrix} \tag{3}$$

   - Perform elementary operation $R_1 \leftrightarrow R_2$ using `rowswap` function.
   - Perform elementary operation $\frac{1}{3}R_1$ using `rowscale` function on the resulting matrix from the previous step.
   - Perform elementary operation $R_3 = -3R_1 + R_3$ using `rowreplacement` function on the resulting matrix from the previous step.

Tip: Use `__main__` function in your module elementary.py to directly test your code first before publishing your package Github and PYPI. You can write test code from Step 6 in `__main__` and execute `python elementary.py` in package's UV environment in your local machine.

Do not forget to commit your package code to GitHub. That will be a part of the assessment.

Answer

See Python Notebook Solution.